

Помоги очень подробно подготовиться к зачету по C++. Нужно разобрать каждый вопрос, каждый подпунктик.

Этот документ охватывает очень объемный и сложный материал по современному C++, многопоточности и структурам данных. Разобрать абсолютно все вопросы в одном сообщении детально невозможно — ответ получится слишком огромным.

PDFЕщё 4

Я предлагаю двигаться по частям. Сейчас я максимально подробно и с примерами разберу **первую тему: «Идеальная передача и вариативные шаблоны»**. Как только вы ее изучите, напишите мне, и мы перейдем ко второму модулю (Многопоточность) и так далее.

PDF

1. Идеальная передача и вариативные шаблоны

PDF

a. lvalue-выражения и rvalue-выражения

PDF

- Что такое выражение? Тип и категория выражения: * *Выражение* (expression) — это последовательность операторов и операндов, которая вычисляет какое-то значение или обозначает какой-то объект/функцию.

PDF

- Каждое выражение в C++ характеризуется двумя свойствами: **типом** (например, `int`, `double`, `std::string`) и **категорией значения** (value category).

- Что такое lvalue и rvalue?

PDF

- **lvalue** (locator value) — это выражение, которое ссылается на конкретную область памяти (имеет адрес). Грубо говоря, это то, что может стоять слева (left) от оператора присваивания. У lvalue есть имя, и его жизненный цикл выходит за рамки одного выражения.
- **rvalue** (read value) — это временное значение, которое не имеет идентифицируемого адреса в памяти. Оно существует только во время вычисления выражения. Это то, что обычно стоит справа (right) от оператора присваивания.

- Примеры:

PDF

- *lvalue*: Имя переменной `int x = 5;` (здесь `x` — это lvalue). Разыменованный указатель `*ptr`. Доступ к элементу массива `arr[0]`.
- *rvalue*: Литералы (например, `5`, `3.14`, кроме строковых литералов,

которые являются lvalue). Результат математической операции `x + 2`.
Результат вызова функции, возвращающей значение (а не ссылку).

b. lvalue-ссылки и rvalue-ссылки

PDF

- Что это такое и в чем разница?

PDF

- **lvalue-ссылка** обозначается как `T&`. Она может привязываться только к lvalue-выражениям (то есть к существующим объектам). Например: `int& ref = x;`
- **rvalue-ссылка** обозначается как `T&&`. Она может привязываться *только* к rvalue-выражениям (временным объектам). Например: `int&& rref = 5;`
- Зачем нужно разделение? Разделение нужно для двух главных концепций современного C++: **семантики перемещения (move semantics)** и **идеальной передачи (perfect forwarding)**. Понимая, что в функцию передали rvalue (временный объект, который скоро исчезнет), мы можем не копировать его ресурсы (например, массив в памяти), а просто "забрать" (переместить) их себе, что работает в разы быстрее.

PDF

- Перегрузка по категории выражения: Вы можете перегрузить функции, чтобы они по-разному реагировали на временные и постоянные объекты:

PDF

```
void printCategory(int& x) { std::cout << "lvalue\n"; }  
void printCategory(int&& x) { std::cout << "rvalue\n"; }
```

- Какую категорию имеет выражение, состоящее только из одного идентификатора rvalue-ссылки? Это важнейший вопрос с подвохом! Если у вас есть переменная-rvalue ссылка (например, внутри функции: `void foo(int&& rref) { ... }`), то само выражение `rref` внутри тела функции является **lvalue**! У него есть имя, и оно хранится в памяти на протяжении всей функции.

PDF

- Что на самом деле делает функция `std::move`? Функция `std::move` **ничего не перемещает**. Она делает только одну вещь: выполняет `static_cast` (приведение типа) переданного ей выражения к rvalue-ссылке (`T&&`). Это просто маркер для компилятора: "Считай этот объект временным, из него можно безопасно украсть ресурсы".

PDF

c.

Универсальные ссылки (Forwarding references)

- Правила вывода типов в шаблонных функциях и правила свёртки (reference collapsing): В C++ запрещены "ссылки на ссылки" (например, `int& &`). Но в шаблонах они могут возникать неявно. Для этого придумали правила свёртки:

PDF

- `T& & → T&`
- `T& && → T&`
- `T&& & → T&`
- `T&& && → T&&` Правило простое: если есть хотя бы одна lvalue-ссылка (&), результат будет lvalue-ссылкой. В противном случае — rvalue-ссылкой.

- Что такое универсальные ссылки и как их написать? Универсальная ссылка (или "forwarding reference") выглядит как rvalue-ссылка (`T&&`), но используется **только в контексте вывода типов шаблона** (или с `auto&&`). Написание: `template <typename T> void foo(T&& arg);`

PDF

- Какой тип выводится при передаче lvalue и rvalue?

PDF

- Если передать **lvalue** (например, `int x`), компилятор выведет тип `T` как `int&`. Из-за правил свёртки `int& &&` превратится в `int&` (lvalue-ссылку).
- Если передать **rvalue** (например, `5`), компилятор выведет тип `T` как `int`. Сигнатура станет `int&&` (rvalue-ссылка). Таким образом, функция "подстраивается" под категорию переданного аргумента.

d.

Типы передачи объекта в функцию

- По значению (копированием): Безопасно, функция имеет свою копию. Недостаток: если объект тяжелый (вектор на миллион элементов), копирование сильно ударит по производительности.

PDF

- По значению (перемещением): Эффективно для передачи владения объектом. Исходный объект останется в валидном, но пустом состоянии.
PDF
- По lvalue-ссылке (T&): Копирования не происходит, можно изменять оригинал. Недостаток: нельзя передать rvalue (временный объект).
PDF
- По rvalue-ссылке (T&&): Используется в основном для реализации конструкторов перемещения и операторов присваивания перемещением.
PDF
- По константной lvalue-ссылке (const T&): Самый популярный способ для чтения. Не копирует объект и принимает как lvalue, так и rvalue. Недостаток: объект нельзя изменять.
PDF
- По универсальной ссылке (T&& в шаблоне): Идеально для написания обёрток, шаблонов и фабрик. Принимает что угодно, сохраняя категорию значения.
PDFЕщё 1

e.

Идеальная передача (Perfect Forwarding)

- Функция `std::forward`, что делает и зачем она нужна? Представьте, что вы написали шаблонную функцию-обертку. Вы хотите передать аргумент из этой обертки в другую функцию точно так же, как он пришел к вам (если пришло rvalue — передать как rvalue, если lvalue — как lvalue). Вспомним, что именованный параметр внутри функции всегда является lvalue. `std::forward<T>(arg)` решает эту проблему: она приводит `arg` к rvalue, *только если* изначально аргумент был передан как rvalue.
PDF
- Отличие `std::forward` от `std::move`: `std::move` безусловно приводит к rvalue. `std::forward` выполняет условное приведение (каст) на основе типа T.
PDF
- Реализация `std::forward` и `std::move`: Под капотом это просто шаблонные функции, использующие `static_cast` и `std::remove_reference` (метафункцию, которая убирает амперсанды из типа, чтобы безопасно кастовать).

f.

Вариативные шаблоны

- Функция с переменным количеством аргументов и шаблонные классы: Позволяют принимать любое количество аргументов любых типов. Синтаксис: `template <typename... Args> void foo(Args... args);`. `Args` здесь — это "пакет параметров шаблона" (parameter pack).
PDFЕщё 1
- Раскрытие пакета и где его можно раскрывать: Раскрытие происходит добавлением многоточия `...`. Это можно делать при вызове функции (`bar(args...);`), при инициализации списком (`std::initializer_list`), при наследовании (`class MyClass : public Bases...`).
PDFЕщё 1
- Выражения свёртки (fold expressions): Появились в C++17. Позволяют применять бинарный оператор ко всем элементам пакета без рекурсии. Например, сумма всех аргументов: `(args + ...)`.
PDF
- Оператор `sizeof...`: Возвращает количество элементов в пакете параметров во время компиляции. Пример: `sizeof...(Args)`.
PDF
- Совместно с идеальной передачей: Самый частый юзкейс (например, `std::make_unique` или `emplace_back` в `std::vector`):
PDF

```
template<typename T, typename... Args>
std::unique_ptr<T> make_unique(Args&&... args) {
    return std::unique_ptr<T>(new
T(std::forward<Args>(args)...));
}
```

Здесь `Args&&... args` — это пакет универсальных ссылок. А `std::forward<Args>(args)...` — это их раскрытие с сохранением категории каждого отдельного аргумента!

Вам все понятно по первому блоку? Жду вашего подтверждения или дополнительных вопросов по этой части, и сразу перейдем к разделу «**2. Основы многопоточного программирования**»!

Давай следующий блок, только не пропускай ни единного вопроса

Отличный настрой! Двигаемся дальше. Этот модуль закладывает фундамент для понимания того, как заставить вашу программу делать несколько дел одновременно. Я пойду строго по пунктам, ничего не упуская.

2. Основы многопоточного программирования.

Потоки

а.

Параллелизм и конкурентность

- **Что такое параллелизм и что такое конкурентность?**

PDF

- **Конкурентность (Concurrency):** Это концепция управления несколькими задачами одновременно. Это когда система может обрабатывать несколько задач, переключаясь между ними (например, на одном ядре процессора). Задачи выполняются "вперемешку", создавая иллюзию одновременности.
- **Параллелизм (Parallelism):** Это физическое выполнение нескольких задач одновременно. Это возможно только на многоядерных или многопроцессорных системах, где каждая задача буквально работает на своем физическом ядре в ту же самую долю секунды.

- **Что такое процесс и что такое поток?**

PDF

- **Процесс:** Это экземпляр выполняемой программы. Операционная система выделяет процессу собственное изолированное адресное пространство (память), ресурсы и потоки ввода/вывода.
- **Поток (Thread):** Это наименьшая единица выполнения *внутри* процесса. Потоки одного процесса делят между собой общую память этого процесса (код, данные, кучу), но имеют собственные стеки и регистры.

- **Организация параллелизма и преимущества/недостатки подходов:**

PDF

- **С использованием процессов:** Мы запускаем несколько независимых программ.
 - **Преимущества:** Высокая безопасность и изоляция (падение одного процесса не "убьет" другой). Легче распределять по сети

(распределенные вычисления).

- *Недостатки:* Долгое создание и переключение контекста (тяжеловесно). Сложное и медленное межпроцессное взаимодействие (IPC — пайпы, сокет, разделяемая память), так как память изолирована.
- **С использованием потоков:** Мы запускаем несколько потоков внутри одного процесса.
 - *Преимущества:* Быстрое создание и переключение контекста (легковесно). Очень быстрое взаимодействие, так как они имеют доступ к одной и той же памяти (глобальным переменным, куче).
 - *Недостатки:* Низкая изоляция (ошибка памяти или исключение в одном потоке может обрушить весь процесс). Главная боль — необходимость сложной синхронизации (мьютексы) для предотвращения гонки данных.

в. Потоки.

Класс `std::thread`

- **Что такое поток (в контексте C++)?** Это объект, который представляет отдельный поток выполнения в ОС. В C++ он управляется через класс `std::thread` из заголовочного файла `<thread>`.

PDF

- **Создание потока:** Вы просто передаете в конструктор `std::thread` функцию (или функтор/лямбду) и её аргументы.

PDF

```
void print_msg(int x) { std::cout << x; }  
std::thread t(print_msg, 5); // Поток начал выполнение
```

- **Методы `join` и `detach`:** У каждого созданного потока вы *обязаны* вызвать один из этих методов до того, как объект потока будет уничтожен.

PDF

- `join()`: Блокирует вызывающий поток (обычно `main`) и ждет, пока поток `t` не завершит свою работу.
- `detach()`: Отсоединяет поток от объекта `std::thread`. Поток продолжает работать в фоновом режиме (демон), и операционная система сама очистит его ресурсы после завершения.

- **Что произойдёт, если выбросится исключение?**

PDF

- **В новом потоке:** Если исключение не перехвачено внутри самого потока (нет `try-catch`), вызывается `std::terminate()`, и **вся программа падает**. Исключение не передается автоматически в родительский поток.
- **В потоке, который создаёт новый поток:** Если родительский поток выбросит исключение *до* того, как вызовет `join()` или `detach()` для созданного потока, то объект `std::thread` будет уничтожен в процессе раскрутки стека. Деструктор `std::thread` проверяет, является ли поток "joinable" (то есть ему не сделали `join/detach`). Если да, деструктор жестко вызывает `std::terminate()`.
- **Передача аргументов:** Аргументы передаются через запятую в конструктор `std::thread`. По умолчанию все аргументы **копируются** в локальную память нового потока.

PDF

С.

Возврат данных из функции потока

- **Использование глобальной переменной:** Самый примитивный способ: один поток пишет в глобальную переменную, другой читает. Это очень опасно, так как требует синхронизации (которую мы рассмотрим в теме мьютексов).

PDF

- **Класс `std::reference_wrapper` и функция `std::ref`:**
`std::reference_wrapper` — это класс-обертка, который ведет себя как ссылка, но при этом является копируемым и присваиваемым объектом (по сути, хранит внутри указатель). `std::ref()` — это вспомогательная функция, которая быстро создает этот объект.

PDF

- **Чем похожи и отличаются от обычных ссылок?**

PDF

- *Похожи:* Их можно неявно преобразовать обратно в обычную ссылку (через `operator T&()`), они используются для изменения оригинального объекта.
- *Отличаются:* Обычную ссылку (`int&`) нельзя переназначить после инициализации. А `std::reference_wrapper` можно переназначать (он Assignable) и копировать (Copyable), что позволяет хранить их в

контейнерах вроде `std::vector` или безопасно передавать в шаблоны (например, в `std::thread`).

- **Почему нельзя использовать обычные ссылки для возврата данных из `std::thread`?** Конструктор `std::thread` — это шаблон. Он берет переданные аргументы и слепо **копирует** их во внутреннюю структуру данных (поток владеет собственными копиями аргументов). Если ваша функция потока ожидает ссылку `void my_func(int& res)`, а конструктор `thread` скопировал ваш `int` и пытается передать эту временную копию (rvalue) в `my_func`, компилятор выдаст ошибку!

PDF

- **Возврат данных с помощью `std::ref`:** Оборачивая аргумент в `std::ref`, мы передаем в конструктор `thread` копируемый объект. Поток копирует эту обертку (которая внутри содержит указатель на оригинал), и при вызове функции `my_func` обертка неявно разворачивается в правильную lvalue-ссылку.

PDFЕщё 3

```
void calc(int& result) { result = 42; }
int res = 0;
std::thread t(calc, std::ref(res)); // Теперь работает!
t.join();
// res теперь равен 42
```

d.

Другое

- **Идентификация потоков:** Каждый поток имеет уникальный ID (тип `std::thread::id`). Узнать его можно:

PDF

- Изнутри самого потока: `std::this_thread::get_id()`
- Снаружи, через объект: `t.get_id()`

- **Передача владения потоком (семантика перемещения):** Объект `std::thread` владеет системным ресурсом. Его нельзя скопировать (как и `std::unique_ptr`), но можно переместить:

PDF

```
std::thread t1(my_func);
std::thread t2 = std::move(t1); // t2 теперь управляет потоком,
t1 пуст
```

- **Создание произвольного количества потоков:** Обычно для этого используют вектор: `std::vector<std::thread> threads;`. В цикле мы можем делать `threads.emplace_back(my_func, args...);`.

PDF

- **Использование стандартных алгоритмов:** Чтобы дождаться завершения всех потоков в векторе, можно использовать стандартный алгоритм:

PDF

```
std::for_each(threads.begin(), threads.end(), [](std::thread& t) {
    if (t.joinable()) t.join();
});
```

- **Функция `std::mem_fn`:** Иногда нужно в качестве функции потока передать *метод класса*. `std::thread` умеет это делать и сам, но `std::mem_fn` (или лямбды) полезны для работы с алгоритмами, если вы хотите вызвать метод у набора объектов. При создании потока с методом класса, первым аргументом передается указатель на метод, а вторым — указатель/ссылка на сам объект: `std::thread t(&MyClass::my_method, &my_object);`

PDF

Мы разобрали жизненный цикл потоков и механику их создания. В следующем блоке мы перейдем к самому "вкусному" — мьютексам и состоянию гонки (когда несколько потоков дерутся за одни данные).

Скажите, вам всё кристально понятно в том, почему деструктор `std::thread` вызывает `std::terminate`, если забыть написать `join()`? Это один из самых частых вопросов на экзаменах!

Следующий блок.

Продолжаем! Третий модуль посвящён одной из главных проблем многопоточности — синхронизации и тому, как не позволить потокам мешать друг другу.

3. Мьютексы

PDF

а.

Состояние гонки

- **Что такое разделяемые данные?** Это данные (переменные, структуры данных, ресурсы), к которым имеют доступ одновременно несколько

потоков. Если потоки только читают данные — проблем нет. Проблемы начинаются, когда хотя бы один поток пытается их изменить.

PDF

- **Что такое состояние гонки (race condition)?** Это ситуация, при которой результат выполнения программы зависит от того, в каком порядке планировщик ОС решит выполнять потоки.

PDF

- *Проблематичные состояния гонки:* Нарушают инварианты (правила) структуры данных. Например, один поток удаляет узел из двусвязного списка, а другой одновременно пытается его прочитать — программа падает.

PDF

- *Безобидные состояния гонки:* Порядок выполнения не влияет на корректность логики программы.

PDF

- **Что такое гонка данных (data race) и к чему она приводит (особенно в C++)?** Гонка данных — это конкретный и очень опасный вид состояния гонки. Она возникает в C++ при соблюдении **трёх условий одновременно** :

PDF

1. Два или более потока обращаются к одной и той же ячейке памяти.
 2. Хотя бы один из потоков выполняет запись (модификацию).
 3. Потоки не используют механизмы синхронизации (например, мьютексы) для упорядочивания этих обращений.
- *К чему приводит:* В стандарте языка C++ гонка данных — это **неопределенное поведение (Undefined Behavior, UB)** ! Это значит, что программа может выдать неверный результат, упасть, или процессор (и компилятор) из-за оптимизаций и барьеров памяти прочтёт "мусорное" значение.

PDF

b.

Стандартный мьютекс

- **Защита данных и класс `std::mutex`:** Мьютекс (от *mutual exclusion* — взаимное исключение) — это примитив синхронизации, который позволяет защитить разделяемые данные. Грубо говоря, это "ключ от туалета":

только один поток может владеть им в один момент времени.

PDF

- **Методы `lock`, `unlock` и `try_lock`:**

PDF

- `lock()`: Поток пытается захватить мьютекс. Если мьютекс уже захвачен другим потоком, текущий поток **блокируется** (засыпает) и ждет, пока мьютекс не освободится.
- `unlock()`: Освобождает мьютекс, позволяя одному из ожидающих потоков захватить его.
- `try_lock()`: Пытается захватить мьютекс. Если он свободен — захватывает и возвращает `true`. Если занят — **не блокирует** поток, а сразу возвращает `false` (поток может пока заняться другими делами).

C.

Стандартные классы `lock_guard` и `unique_lock`

- **В чём недостатки класса `std::mutex`?** Главный недостаток: нужно не забыть вызвать `unlock()`. Если вы напишете `mutex.lock()`, а затем в коде произойдет ранний `return` или будет выброшено исключение (exception), до `mutex.unlock()` выполнение не дойдет. Мьютекс останется заблокированным навсегда, и программа зависнет.

PDF

- **Класс `std::lock_guard` и его преимущества:** Это простая обёртка, реализующая идиому RAII (Resource Acquisition Is Initialization). Вы передаете ей мьютекс в конструкторе (она вызывает `lock()`), а в деструкторе (когда обертка выходит из области видимости) она автоматически вызывает `unlock()`.

PDF

- *Преимущество перед `std::mutex`:* Полная безопасность относительно исключений (exception safety) и защита от забывчивости программиста

.

PDF

- **Класс `std::unique_lock`, его преимущества и недостатки перед `lock_guard`:** Это "старший брат" `lock_guard`.

PDF

- *Преимущества:* Гибкость. Вы можете не захватывать мьютекс сразу в конструкторе (передав `std::defer_lock`), можете вручную вызывать

у него `lock()` и `unlock()`, можете передавать владение мьютексом другому `unique_lock` (через `std::move`), и он работает с условными переменными.

- *Недостатки:* За гибкость нужно платить. `unique_lock` внутри хранит `bool` флаг (захвачен мьютекс или нет), поэтому он занимает чуть больше памяти и работает самую малость медленнее, чем `lock_guard` (которому флаг не нужен, так как он всегда захвачен).

PDF

d.

Взаимоблокировка

- **Что такое взаимоблокировка (deadlock)?** Это ситуация, когда два (или более) потока вечно ждут друг друга. Например: Поток 1 захватил Мьютекс А и ждет Мьютекс Б. Поток 2 захватил Мьютекс Б и ждет Мьютекс А. Никто не может продолжить работу.

PDFЕщё 1

- **Решение с помощью `std::lock`:** Если вам нужно захватить сразу два или более мьютексов, используйте стандартную функцию `std::lock(m1, m2, ...)`. Она использует специальный алгоритм предотвращения дедлоков: захватывает все переданные мьютексы "атомарно" (без риска, что другой поток встрянет между захватами в обратном порядке).

PDFЕщё 1

e.

Защита разделяемых данных во время инициализации

PDFЕщё 4

Иногда данные нужно инициализировать только один раз (например, создание Singleton-объекта или открытие файла на чтение), а дальше потоки будут только читать их. Захватывать обычный мьютекс при каждом чтении — слишком медленно.

- **Паттерн блокировка с двойной проверкой (double check locking):** Суть паттерна: сначала проверяем указатель без блокировки. Если он `null`, берем мьютекс, снова проверяем (вдруг другой поток уже создал объект, пока мы брали мьютекс) и создаем. *Важно:* в C++ до появления C++11 этот паттерн был сломан из-за переупорядочивания инструкций

процессором (могли прочитать "полуготовый" объект).

PDF

- **Класс `std::once_flag` и функция `std::call_once`:** Это стандартное, потокобезопасное решение проблемы в современном C++. Вы создаете объект `std::once_flag`. Затем потоки вызывают `std::call_once(flag, my_init_function)`. Стандарт гарантирует, что функция выполнится **ровно один раз**. Если несколько потоков вызовут `call_once` одновременно, один будет выполнять инициализацию, а остальные будут заблокированы до её завершения.

PDFЕщё 1

Следующий блок

Переходим к четвёртому модулю. Здесь мы рассмотрим более высокоуровневые инструменты для координации потоков, чтобы они не просто блокировали друг друга, а могли "общаться" и передавать результаты своей работы.

PDF

4. Механизмы синхронизации

PDF

а.

Условные переменные

- **Что такое условные переменные и класс `std::condition_variable`:** Условная переменная — это механизм, который позволяет потоку заснуть (приостановить выполнение) и ждать, пока другой поток не сообщит ему, что произошло какое-то событие (или выполнилось условие). `std::condition_variable` работает **только** в связке с `std::unique_lock<std::mutex>`.

PDF

- **Как им пользоваться и метод `wait`:** Когда поток вызывает `wait(lock)`, под капотом происходит магия:
 1. Метод `wait` атомарно **освобождает мьютекс** и отправляет текущий поток в сон.
 2. Пока поток спит, другие потоки могут захватить мьютекс и изменить общие данные.
 3. Когда поток просыпается (после уведомления), `wait` **снова захватывает мьютекс** перед тем, как вернуть управление вашей

PDF

программе.

PDF

- **Методы `notify_one` и `notify_all`:** Другой поток, который изменил данные (например, добавил элемент в очередь), вызывает эти методы:
 - `notify_one()`: Будит ровно один случайно выбранный поток, который сейчас спит на `wait`.PDF
 - `notify_all()`: Будит вообще все потоки, ожидающие на этой условной переменной. (Они проснутся, но начнут по очереди пытаться захватить мьютекс).
- PDF

- Ложные пробуждения (spurious wake): Это специфика работы операционных систем. Поток, спящий на `wait`, может внезапно проснуться сам по себе, даже если никто не вызывал `notify_!`

PDFЕщё 1

- *Как бороться:* `wait` всегда нужно оборачивать в цикл `while (!условие) { cv.wait(lock); }`. В C++ есть удобная перегрузка `wait`, принимающая предикат (лямбду): `cv.wait(lock, []{ return data_ready; })`; . Она сама крутит этот цикл под капотом.

b.

Запуск асинхронной задачи

- **Функция `std::async`:** Это высокоуровневая обёртка над потоками. Вместо того чтобы вручную создавать `std::thread`, вы просто передаете функцию в `std::async`. Компилятор и стандартная библиотека сами решат, запустить ли её в новом потоке прямо сейчас или выполнить позже (отложено), хотя вы можете управлять этим с помощью флагов `std::launch::async` или `std::launch::deferred`.

PDF

- **Возврат значения с помощью `std::future`:** Когда вы вызываете `std::async`, она немедленно возвращает объект `std::future`. Это своего рода "билетик", по которому вы позже сможете получить результат.

PDFЕщё 1

- Когда вам нужен результат, вы вызываете метод `future.get()`.
- Если асинхронная задача уже завершилась, `get()` сразу вернет значение. Если еще выполняется — `get()` заблокирует текущий поток

и будет ждать её завершения.

c.

Класс `packaged_task`

- **Что такое `std::packaged_task`:** Этот класс связывает любую вызываемую сущность (функцию, лямбду) с объектом `std::future`.
PDF
- **Зачем он нужен?** Он позволяет *разделить* моменты создания задачи и её выполнения. Например, вы можете упаковать задачу в `packaged_task`, положить её в очередь (что часто делают при создании пулов потоков), а какой-нибудь свободный рабочий поток позже достанет её и выполнит.
PDF
- **Методы `get_future()` и `operator()`:**
 - `get_future()`: Возвращает объект `future`, из которого потом можно будет прочитать результат выполнения этой задачи.
PDF
 - `operator()`: Собственно, сам запуск задачи. Вызов этого оператора выполняет упакованную функцию и автоматически записывает результат (или выброшенное исключение) во внутреннее общее состояние, которое связано с `future`.
PDF
- Передача `packaged_task`: `packaged_task` нельзя скопировать (она владеет уникальным состоянием), но её можно перемещать (с помощью `std::move`). Именно так её передают в другие потоки или помещают в контейнеры.
PDFЕщё 1

d.

Класс `promise`

PDFЕщё 3

- **Что такое `std::promise`:** Если `future` — это "получатель" результата, то `promise` — это "отправитель". Это самый низкоуровневый способ передать значение из одного потока в другой без использования глобальных

переменных и мьютексов вручную.

PDF

- Методы `get_future`, `set_value` и `set_exception`:

PDF

- `get_future()`: Создает и возвращает связанный объект `future`. Вы отдаете этот `future` потоку-читателю.

PDF

- `set_value(val)`: В потоке-писателе вы вызываете этот метод, чтобы записать результат. В этот момент `future` в другом потоке становится "готовым", и его метод `get()` разблокируется, возвращая `val`.

PDF

- `set_exception(ptr)`: Если в потоке-работнике что-то пошло не так, вы перехватываете исключение и передаете его в `promise`. Когда поток-читатель вызовет `future.get()`, это исключение будет "распаковано" и выброшено в потоке-читателе!

PDF

Следующий блок

Пятый модуль — это логичное применение всех знаний из предыдущих блоков. Теперь мы будем использовать мьютексы и условные переменные, чтобы заставить классические структуры данных работать в многопоточной среде безопасно.

PDF

5. Потокобезопасные стек и очередь с блокировками

а. Потокобезопасные структуры данных

- **Что такое потокобезопасная структура данных?** Это структура данных, методы которой могут быть вызваны одновременно из нескольких потоков без возникновения состояния гонки (data race) или неопределенного поведения, при этом инварианты (правила) структуры данных не нарушаются.

PDF

- **Что такое потокобезопасная структура данных с блокировками?** Это структура, внутри которой для синхронизации доступа потоков используются мьютексы (блокировки). Если один поток изменяет

структуру, другие потоки блокируются и ждут своей очереди.

PDF

- **Являются ли стандартные контейнеры STL потокобезопасными?** Нет, стандартные контейнеры (`std::vector`, `std::list`, `std::map` и т.д.) **не являются** потокобезопасными для записи. Стандарт C++ гарантирует только то, что несколько потоков могут одновременно *читать* из одного и того же контейнера. Но если хотя бы один поток пишет, а другие читают (или тоже пишут) без внешней синхронизации, это приведет к гонке данных.

PDF

b. Недостатки стандартного класса `std::stack`

- **Стандартный класс `std::stack` и его методы:** `std::stack` работает по принципу LIFO (последним пришел — первым ушел). Основные методы: `push(val)` (добавить на вершину), `top()` (получить ссылку на верхний элемент) и `pop()` (удалить верхний элемент, **ничего не возвращая**).

PDF

- **В чём недостатки интерфейса для многопоточности?** Интерфейс `std::stack` порождает классическое состояние гонки. Представьте код:

```
if (!s.empty()) { auto val = s.top(); s.pop(); }
```

В многопоточной среде между вызовами `top()` и `pop()` может вклиниться другой поток и удалить элемент. В итоге первый поток прочитает не тот элемент или, что хуже, вызовет `top()` у уже пустого стека, что приведет к крашу.

PDF

- **Почему в стандартной библиотеке C++ стек реализован именно так?** Из-за **безопасности относительно исключений (exception safety)**. Если бы метод `pop()` возвращал элемент по значению (т.е. `T pop()`), то при возврате из функции мог бы вызваться конструктор копирования типа `T`. Если этот конструктор копирования выбросит исключение (например, не хватило памяти), то элемент будет уже удален из стека, но так и не будет доставлен вызывающему коду. Данные будут безвозвратно утеряны! Поэтому `top()` только читает, а `pop()` только удаляет.

PDF

c. Потокобезопасный стек с блокировками

- **Реализация на основе `std::stack`:** Мы создаем класс-обертку, который

содержит внутри себя `std::stack` и `std::mutex`.

PDF

- **Реализация методов `push` и `pop`:** Метод `push` просто захватывает мьютекс через `std::lock_guard` и вызывает метод `push` внутреннего стека. Метод `pop` должен объединить стандартные `top()` и `pop()` в одну атомарную операцию.

PDF

- **Безопасность относительно исключений для такого стека:** Чтобы избежать потери данных при объединении `top()` и `pop()`, мы не можем просто возвращать элемент по значению. Есть два безопасных варианта реализации потокобезопасного `pop`:

PDF

1. Передавать ссылку на результат в параметры функции: `void pop(T& result)`. Значение копируется в `result` до того, как элемент будет удален из стека.
2. Возвращать умный указатель (обычно `std::shared_ptr<T>`). Конструктор копирования `shared_ptr` не бросает исключений, поэтому возврат безопасен.

d. Потокобезопасная очередь с блокировками

- **Интерфейс очереди и её реализация:** Очередь работает по принципу FIFO (первым пришел — первым ушел). Мы также оборачиваем `std::queue`, `std::mutex` и добавляем `std::condition_variable` для эффективного ожидания данных.

PDF

- **Реализация `push`, `try_pop` и `wait_and_pop`:**

- `push(val)`: Блокирует мьютекс, добавляет элемент в очередь и вызывает `cv.notify_one()`, чтобы разбудить ожидающий поток.

PDF

- `try_pop(T& result)`: Блокирует мьютекс. Если очередь пуста, немедленно возвращает `false`. Если нет — копирует верхний элемент в `result`, удаляет его из очереди и возвращает `true`.

PDF

- `wait_and_pop(T& result)`: Блокирует мьютекс с помощью `std::unique_lock` и вызывает `cv.wait(lock, []{ return !q.empty(); })`. Поток спит, пока очередь пуста. Как только кто-то делает `push`, поток просыпается, забирает элемент и удаляет его.

- **Безопасность относительно исключений:** Здесь применяются те же правила, что и для стека — методы `pop` должны либо принимать ссылку для записи результата, либо возвращать `std::shared_ptr`.

PDF

е. Потокбезопасная очередь с блокировками на основе односвязного списка

- **Использование двух мьютексов (защита головы и хвоста):** Очередь из предыдущего пункта имеет "узкое место": методы `push` и `pop` блокируют один и тот же мьютекс. Это значит, что добавление и извлечение не могут происходить одновременно. Можно ускорить очередь, используя структуру односвязного списка и **два мьютекса**: `head_mutex` для метода `pop` (забираем с начала) и `tail_mutex` для `push` (добавляем в конец).

PDF

- **Реализация и фиктивный узел (dummy node):** Чтобы `push` и `pop` никогда не конфликтовали за один и тот же узел (что произошло бы, если бы в очереди был только один элемент), очередь всегда инициализируется с одним фиктивным узлом.
 - `tail` всегда указывает на этот фиктивный узел.
 - Когда происходит `push`, мы записываем данные в текущий фиктивный узел (на который указывает `tail`), создаем новый фиктивный узел, привязываем его к текущему и сдвигаем указатель `tail` на новый фиктивный узел.
 - Метод `pop` работает только с указателем `head`. Это обеспечивает независимую работу алгоритмов (Fine-grained locking).
- **Безопасность относительно исключений:** Создание нового узла для `push` происходит **до** захвата мьютекса или переназначения указателей. Если оператор `new` выбросит исключение (например `std::bad_alloc`), то структура очереди никак не пострадает, что делает такую реализацию строго безопасной.

PDFЕщё 1

Следующий блок

Шестой модуль опускает нас на более низкий уровень. Мы уходим от "тяжелых" объектов ОС вроде мьютексов и смотрим, как работают многопоточность и память на уровне железа и компилятора. Это самая сложная, но невероятно интересная часть C++!

6. Модели памяти

а. Барьеры памяти

- **Причина неопределённого поведения (UB) при гонке данных:**

Компилятор и процессор по умолчанию исходят из того, что ваш код выполняется в одном потоке. Чтобы код работал быстрее, они безжалостно его оптимизируют: кэшируют переменные в регистрах и меняют порядок команд. Если два потока одновременно трогают одни и те же данные без синхронизации, эти "однопоточные" оптимизации ломают логику программы.

PDFЕщё 1

- **Когерентность кэша:** У каждого ядра процессора есть свой сверхбыстрый кэш (L1/L2). Если Поток 1 (на Ядре 1) изменяет переменную `x`, он может записать новое значение только в свой кэш, а не в основную (RAM) память. Поток 2 (на Ядре 2) при чтении `x` прочитает старое значение из своего кэша. За синхронизацию этих кэшей отвечает протокол когерентности кэшей (например, MESI), но это происходит не мгновенно.

PDFЕщё 1

- **Переупорядочивание инструкций:** Процессор (благодаря Out-of-Order execution) и компилятор могут менять местами инструкции, если не видят между ними зависимости в рамках *одного* потока. Например:

PDF

```
int data = 42;    // Инструкция 1
bool ready = true; // Инструкция 2
```

Процессор может решить сначала выполнить Инструкцию 2, а потом Инструкцию 1. Для текущего потока разницы нет. Но другой поток может увидеть `ready == true`, прочитать `data`, а там окажется мусор (потому что Инструкция 1 еще не выполнилась)!

PDF

- **Барьеры памяти (Memory Barriers):** Это специальные инструкции процессора, которые буквально говорят: "Стой! Пока всё, что было до этой линии, не запишется в память, дальше не идем". Они запрещают переупорядочивание.

PDFЕщё 1

- **Виды барьеров:**

- **LoadLoad:** Операции чтения (Load) до барьера выполняются строго до

операций чтения после барьера .

PDF

- **StoreStore:** Операции записи (Store) до барьера выполняются до записей после него .

PDF

- **LoadStore / StoreLoad:** Запрещают перестановку соответствующих операций . `StoreLoad` — самый "тяжелый" барьер.

PDF

- **Acquire (Получение):** Обычно ставится при чтении. Запрещает любым последующим операциям чтения/записи "перепрыгнуть" барьер и выполняться *до* него .

PDF

- **Release (Освобождение):** Обычно ставится при записи. Запрещает любым предыдущим операциям "перепрыгнуть" барьер и выполняться *после* него .

PDF

b. Модели памяти в языке C++

- **Упорядочение доступа к памяти (Memory Order):** C++11 стандартизировал эти аппаратные барьеры в виде аргументов для атомарных операций .

PDF

- `memory_order_seq_cst` (Sequential Consistency): **По умолчанию.** Самый строгий режим. Гарантирует, что абсолютно все потоки увидят изменения всех переменных в одном и том же порядке . Безопасно, но медленно.

PDF

- `memory_order_acquire` и `memory_order_release`: Работают в паре. Запись с `release` в Потоке А синхронизируется с чтением с `acquire` в Потоке Б . Всё, что Поток А записал до `release`, будет гарантированно видно Поток Б после `acquire` .

PDFЕщё 1

- `memory_order_relaxed`: Никакой синхронизации и барьеров нет . Гарантируется только то, что сама операция над переменной будет атомарной . Используется, например, для простых счетчиков статистики.

PDFЕщё 1

- **Функция `std::atomic_thread_fence`:** Позволяет установить барьер памяти вручную, не привязывая его к конкретной атомарной операции (например, `std::atomic_thread_fence(std::memory_order_release);`).
PDF

с. Атомарные типы и операции над ними

- **Отличие атомарных переменных от обычных:** Операции над обычными переменными (даже `x++`) состоят из нескольких шагов (прочитать из памяти, увеличить, записать обратно). Другой поток может вклиниться между этими шагами. Атомарные переменные (из `<atomic>`) гарантируют, что операция неделима (выполняется за один такт/инструкцию процессора), и никто не увидит промежуточного состояния. Они не вызывают гонок данных.

PDFЕщё 3

- **Класс `std::atomic_flag` и его методы:** Это простейший атомарный булев тип. Его главная особенность — это **единственный тип в C++, для которого стандарт строго гарантирует, что он работает без неявных блокировок (lock-free)** на любой платформе.

PDF

- `test_and_set()`: Атомарно устанавливает флаг в `true` и возвращает его **предыдущее** значение.

PDF

- `clear()`: Атомарно сбрасывает флаг в `false`.

PDF

- **Атомарные типы `atomic<T>` и методы:**

- `load()`: Атомарно читает значение.

PDF

- `store(val)`: Атомарно записывает значение.

PDF

- `compare_exchange_weak` / `compare_exchange_strong` (CAS — Compare And Swap): Самая важная операция для неблокирующих алгоритмов. Смысл: "Если текущее значение переменной равно `expected`, замени его на `desired` и верни `true`. Если не равно — ничего не меняй, но запиши в `expected` то, что там лежит на самом деле, и верни `false`".

PDFЕщё 1

- **Реализация спинлока (spinlock) на основе атомарной переменной:**

Спинлок — это мьютекс, который не усыпляет поток (что требует долгого вызова ОС), а заставляет его крутиться в пустом цикле, сжигая CPU, пока мьютекс не освободится. Идеально для очень коротких блокировок.

PDFЕщё 1

```
class Spinlock {
    std::atomic_flag flag = ATOMIC_FLAG_INIT;
public:
    void lock() {
        // Пока test_and_set возвращает true (кто-то другой
        // держит лок),
        // мы крутимся в цикле (spin).
        // memory_order_acquire гарантирует, что мы увидим
        // актуальные данные после захвата.
        while (flag.test_and_set(std::memory_order_acquire))
        {
            // можно добавить __mm_pause() для оптимизации
            // энергопотребления
        }
    }

    void unlock() {
        // Сбрасываем флаг в false.
        // memory_order_release гарантирует, что все наши записи
        // завершены до снятия лока.
        flag.clear(std::memory_order_release);
    }
};
```

Следующий блок

Мы на финишной прямой! Оставшиеся седьмой и восьмой модули — это высший пилотаж (lock-free программирование и пулы потоков). Здесь мы отказываемся от мьютексов ради максимальной скорости.

7. Потокобезопасные стек и очередь без блокировок

PDF

a.

Основные определения

Алгоритмы без блокировок (lock-free) не используют мьютексы, а опираются на атомарные операции (в первую очередь CAS — `compare_exchange`). Это

защищает от дедлоков, но требует очень сложной логики.

- **Неблокирующие структуры данных (Non-blocking):** Это общий термин. Он означает, что приостановка (или сбой) одного потока никогда не приведет к приостановке других потоков, работающих с этой же структурой. (В отличие от мьютекса: если поток захватил мьютекс и "умер", все остальные зависнут навсегда).

PDF

- **Свободные от блокировок (Lock-free):** Структура гарантирует, что *хотя бы один* поток в системе сделает полезную работу за конечное число шагов. Если несколько потоков одновременно пытаются сделать `push`, они будут "толкаться", и один обязательно преуспеет, а остальные попытаются снова в цикле.

PDFЕщё 1

- **Свободные от ожидания (Wait-free):** Самый строгий класс. Гарантирует, что *каждый* поток выполнит свою операцию за конечное число шагов, независимо от действий других. Это идеальный сценарий для систем реального времени, но написать такой код на C++ невероятно сложно.

PDF

b.

Реализация потокобезопасного стека без блокировок

Основа lock-free стека — атомарный указатель на вершину:

```
std::atomic<Node*> head; .
```

PDF

- **Метод `push`:** Мы создаем новый узел (`new_node`). В цикле `do-while` делаем:

1. Читаем текущую голову: `new_node->next = head.load()`.
2. Выполняем CAS: `head.compare_exchange_weak(new_node->next, new_node)`. Если за время шага 1 никто не изменил голову, мы атомарно ставим `new_node` на место `head`. Если изменил — цикл повторяется.

PDF

- **Метод `pop`:** Читаем `old_head = head.load()`. В цикле: `head.compare_exchange_weak(old_head, old_head->next)`. Если успешно, мы отсоединили узел от стека.

- **Почему в базовой версии есть утечки памяти?** Если мы просто сделаем

`delete old_head`, возникнет фатальная проблема (ABA problem/гонка за указатель). Пока наш поток №1 спал перед выполнением CAS, поток №2 мог удалить `old_head` и вернуть память ОС. Пробудившись, поток №1 обратится к удаленной памяти (`old_head->next`) и получит *Segmentation Fault*. Поэтому в простейшей версии память просто не освобождают.

PDFЕщё 1

С.

Управление памятью в структурах данных без блокировок

PDFЕщё 2

Чтобы стек не потреблял бесконечное количество ОЗУ, память нужно очищать. Но удалять узел можно *только тогда*, когда ни один другой поток его не читает.

PDF

- **Метод подсчёта количества потоков:** Вводится атомарный счетчик потоков, находящихся внутри функции `pop`.

PDF

- Если счетчик равен 1 (наш поток — единственный), мы можем безопасно вызвать `delete`.

PDF

- Если счетчик > 1 , мы кладем узел в список "на удаление" (`to_be_deleted`). Этот список очистит тот поток, который выйдет из `pop` последним (когда счетчик станет 0).

PDF

- *Недостаток:* При высокой конкуренции счетчик почти никогда не опускается до 1, и список мусора бесконечно растет.

PDF

- **Метод указателей опасности (Hazard Pointers):** Глобальный массив атомарных указателей.

PDF

- Перед тем как обратиться к узлу, поток записывает его адрес в свою ячейку массива Hazard Pointers (анонсирует: "Этот узел опасен, я с ним работаю!").
- При `pop` узел добавляется в локальный список мусора потока.
- Периодически поток сканирует все чужие Hazard Pointers. Если адрес узла из мусора нигде не встречается (он безопасен), делается `delete`.

PDF

- **Преимущество:** Работает быстро и гарантированно чистит память, но очень сложен в реализации.

PDF

- **Очередь без блокировок (с устранением утечек):** Имеет `head` и `tail`. Логика та же, но для избежания конфликтов при `push` и `pop` одновременно пустого узла, алгоритм требует фиктивного узла и вспомогательных функций для "подтягивания" хвоста другими потоками. Память управляется через `Hazard Pointers`.

PDFЕщё 2

8. Пул потоков

PDF

Создание объектов `std::thread` — дорогая операция ОС. Если у вас 10 000 коротких задач, создавать 10 000 потоков бессмысленно и медленно.

- **Что такое пул потоков и его реализация?** Пул потоков (Thread Pool) — это набор заранее созданных "рабочих" потоков, которые спят и ждут поступления задач.

PDF

- Реализуется с помощью вектора потоков `std::vector<std::thread>`, потокобезопасной очереди задач (например, `std::function<void()>`) и условной переменной.

PDF

- Рабочие потоки крутятся в бесконечном цикле, вызывая `wait_and_pop` у очереди задач. Получив задачу, выполняют её и снова просят новую.

PDF

- **Ожидание задачи, переданной пулу:** Чтобы получить результат из пула, задачу обычно заворачивают в `std::packaged_task`. Функция `submit` (отправки задачи в пул) возвращает пользователю `std::future`. В основном потоке мы делаем `future.get()` и ждем завершения.

PDF

- **Предотвращение конкуренции за очередь работ:** Если в пуле 16 ядер (потоков), и все они дергают `pop` у одной глобальной очереди с одним мьютексом, то они будут блокировать друг друга. Глобальная очередь становится "бутылочным горлышком".

PDF

- **Занимание работ (Work Stealing):** Изящное решение проблемы конкуренции.

PDF

- Вместо одной большой очереди, **каждому** рабочему потоку выдается своя *локальная* очередь задач .

PDF

- Поток забирает задачи только из своей очереди (без мьютексов или с минимальной синхронизацией) .

PDF

- Если локальная очередь потока опустела, он идет в очередь к другому (соседнему) потоку и **ворует (steals)** задачу с конца его очереди (чтобы не мешать владельцу, который берет с начала) . Это равномерно распределяет нагрузку на все ядра .

PDFЕщё 1

Поздравляю! Мы разобрали абсолютно все вопросы из документа, от rvalue-ссылок до Hazard-указателей и пулов потоков.

Скажите, как вы планируете всё это повторять? Хотите ли вы, чтобы я устроил вам небольшую "тренировочную проверку" (задавал случайные вопросы по этим темам, как преподаватель на экзамене), или вам нужно сгенерировать краткие шпаргалки по каким-то конкретным пунктам?